

Structure and File Management

overview

- A data structure is a group of data elements grouped together under one name. These data elements, known as members, can have different types and different lengths. Data structures can be declared in C++ using the following syntax:

```
struct type_name {  
    member_type1 member_name1;  
    member_type2 member_name2;  
    member_type3 member_name3;  
} object_names;
```

- Where type_name is a name for the structure type, object_name can be a set of valid identifiers for objects that have the type of this structure. Within braces {}, there is a list with the data members, each one is specified with a type and a valid identifier as its name.

Example

```
struct product {  
    int weight;  
    double price;  
};  
product apple;  
product banana, melon;
```

- This declares a structure type, called product, and defines it having two members: weight and price, each of a different fundamental type. This declaration creates a new type (product), which is then used to declare three objects (variables) of this type: apple, banana, and melon. Note how once product is declared, it is used just like any other type.

- Right at the end of the struct definition, and before the ending semicolon (;), the optional field `object_names` can be used to directly declare objects of the structure type. For example, the structure objects `apple`, `banana`, and `melon` can be declared at the moment the data structure type is defined

```
struct product {  
    int weight;  
    double price;  
} apple, banana, melon;
```

- In this case, where `object_names` are specified, the type name (`product`) becomes optional: `struct` requires either a `type_name` or at least one name in `object_names`, but not necessarily both.

- Once the three objects of a determined structure type are declared (apple, banana, and melon) its members can be accessed directly. The syntax for that is simply to insert a dot (.) between the object name and the member name. For example, we could operate with any of these elements as if they were standard variables of their respective types:

apple.weight

apple.price

banana.weight

banana.price

melon.weight

melon.price

- Each one of these has the data type corresponding to the member they refer to: apple.weight, banana.weight, and melon.weight are of type int, while apple.price, banana.price, and melon.price are of type double.

Pointers to structures

- Like any other type, structures can be pointed to by its own type of pointers:

```
struct movies_t {  
    string title;  
    int year;  
};  
movies_t amovie;  
movies_t * pmovie;
```

- Here amovie is an object of structure type movies_t, and pmovie is a pointer to point to objects of structure type movies_t. Therefore, the following code would also be valid:

```
pmovie = &amovie;
```

- The value of the pointer `pmovie` would be assigned the address of object `amovie`.
- The arrow operator (`->`) is a dereference operator that is used exclusively with pointers to objects that have members. This operator serves to access the member of an object directly from its address.

`pmovie->title`

- is, for all purposes, equivalent to:
`(*pmovie).title`
- Both expressions, `pmovie->title` and `(*pmovie).title` are valid, and both access the member `title` of the data structure pointed by a pointer called `pmovie`. It is definitely something different than: `*pmovie.title`
- which is rather equivalent to: `*(pmovie.title)`

- This would access the value pointed to by a hypothetical pointer member called title of the structure object pmovie (which is not the case, since title is not a pointer type). The following panel summarizes possible combinations of the operators for pointers and for structure members:

Expression	What is evaluated	Equivalent
a.b	Member b of object a	
a->b	Member b of object pointed to by a	(*a).b
*a.b	Value pointed to by member b of object a	*(a.b)

Nesting structures

- Structures can also be nested in such a way that an element of a structure is itself another structure:

```
struct movies_t {  
    string title;  
    int year;  
};  
struct friends_t {  
    string name;  
    string email;  
    movies_t favorite_movie;  
} charlie, maria;  
friends_t * pfriends = &charlie;
```

all of the following expressions would be valid:

charlie.name

maria.favorite_movie.title

charlie.favorite_movie.year

pfriends->favorite_movie.year

(the last two expressions refer to the same member).

File Management

Input/output with files

- C++ provides the following classes to perform output and input of characters to/from files:
 - ofstream: Stream class to write on files
 - ifstream: Stream class to read from files
 - fstream: Stream class to both read and write from/to files.
- These classes are derived directly or indirectly from the classes istream and ostream.
- we can use our file streams the same way we use cin and cout, with the only difference that we have to associate these streams with physical files.

Example

```
// basic file operations
#include <iostream>
#include <fstream>
using namespace std;
int main () {
    ofstream myfile;
    myfile.open ("example.txt");
    myfile << "Writing this to a file.\n";
    myfile.close();
    return 0;
}
```

- This code creates a file called example.txt and inserts a sentence into it in the same way we used to do with cout, but using the file stream myfile instead.

Open a file

- The first operation generally performed on an object of one of these classes is to associate it to a real file. This procedure is known as to open a file. An open file is represented within a program by a stream (i.e., an object of one of these classes; in the previous example, this was myfile) and any input or output operation performed on this stream object will be applied to the physical file associated to it.
- In order to open a file with a stream object we use its member function open:

open (filename, mode);

- Where filename is a string representing the name of the file to be opened, and mode is an optional parameter with a combination of the following flags:

ios::in	Open for input operations.
ios::out	Open for output operations.
ios::binary	Open in binary mode.
ios::ate	Set the initial position at the end of the file. If this flag is not set, the initial position is the beginning of the file.
ios::app	All output operations are performed at the end of the file, appending the content to the current content of the file.
ios::trunc	If the file is opened for output operations and it already existed, its previous content is deleted and replaced by the new one.

- All these flags can be combined using the bitwise operator OR (|). For example, if we want to open the file example.bin in binary mode to add data we could do it by the following call to member function open:

```
ofstream myfile;
```

```
myfile.open ("example.bin", ios::out | ios::app | ios::binary);
```

- Each of the open member functions of classes ofstream, ifstream and fstream has a default mode that is used if the file is opened without a second argument:

class	default mode parameter
ofstream	ios::out
ifstream	ios::in
fstream	ios::in ios::out

- For ifstream and ofstream classes, ios::in and ios::out are automatically and respectively assumed, even if a mode that does not include them is passed as second argument to the open member function (the flags are combined).
- For fstream, the default value is only applied if the function is called without specifying any value for the mode parameter. If the function is called with any value in that parameter the default mode is overridden, not combined.
- File streams opened in binary mode perform input and output operations independently of any format considerations. Non-binary files are known as text files, and some translations may occur due to formatting of some special characters (like newline and carriage return characters).

- Since the first task that is performed on a file stream is generally to open a file, these three classes include a constructor that automatically calls the `open` member function and has the exact same parameters as this member. Therefore, we could also have declared the previous `myfile` object and conduct the same opening operation in our previous example by writing:

```
ofstream myfile ("example.bin", ios::out | ios::app | ios::binary);
```

- Combining object construction and stream opening in a single statement. Both forms to open a file are valid and equivalent.
- To check if a file stream was successful opening a file, you can do it by calling to member `is_open`. This member function returns a `bool` value of `true` in the case that indeed the stream object is associated with an open file, or `false` otherwise:

```
if (myfile.is_open()) { /* ok, proceed with output */ }
```

Closing a file

- When we are finished with our input and output operations on a file we shall close it so that the operating system is notified and its resources become available again. For that, we call the stream's member function `close`. This member function takes flushes the associated buffers and closes the file:

```
myfile.close();
```

- Once this member function is called, the stream object can be re-used to open another file, and the file is available again to be opened by other processes.
- In case that an object is destroyed while still associated with an open file, the destructor automatically calls the member function `close`.

Text files

- Text file streams are those where the `ios::binary` flag is not included in their opening mode. These files are designed to store text and thus all values that are input or output from/to them can suffer some formatting transformations, which do not necessarily correspond to their literal binary value.
- Writing operations on text files are performed in the same way with `cout`:

```
int main () {  
    ofstream myfile ("example.txt");  
    if (myfile.is_open())  
    {  
        myfile << "This is a line.\n";  
        myfile << "This is another line.\n";  
        myfile.close();  
    }
```

```
    else cout << "Unable to open file";  
    return 0;  
}
```

reading

- Reading from a file can also be performed in the same way that we did with cin:

```
int main () {  
    string line;  
    ifstream myfile ("example.txt");  
    if (myfile.is_open())  
    {  
        while ( getline (myfile,line) )  
        {  
            cout << line << '\n';  
        }  
        myfile.close();  
    }  
    else cout << "Unable to open file";  
    return 0;  
}
```

This last example reads a text file and prints out its content on the screen. We have created a while loop that reads the file line by line, using `getline`. The value returned by `getline` is a reference to the stream object itself, which when evaluated as a boolean expression (as in this while-loop) is true if the stream is ready for more operations, and false if either the end of the file has been reached or if some other error occurred.

Checking state flags

- The following member functions exist to check for specific states of a stream (all of them return a bool value):
- `bad()`: Returns true if a reading or writing operation fails. For example, in the case that we try to write to a file that is not open for writing or if the device where we try to write has no space left.
- `fail()`: Returns true in the same cases as `bad()`, but also in the case that a format error happens, like when an alphabetical character is extracted when we are trying to read an integer number.
- `eof()`: Returns true if a file open for reading has reached the end.
- `good()`: It is the most generic state flag: it returns false in the same cases in which calling any of the previous functions would return true. Note that `good` and `bad` are not exact opposites (`good` checks more state flags at once).
- The member function `clear()` can be used to reset the state flags.

Dynamic memory

- previously, all memory needs were determined before program execution by defining the variables needed. But there may be cases where the memory needs of a program can only be determined during runtime. For example, when the memory needed depends on user input.
- On these cases, programs need to dynamically allocate memory, for which the C++ language integrates the operators `new` and `delete`.

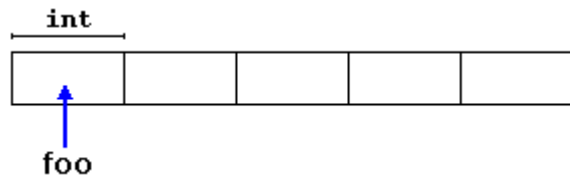
- Dynamic memory is allocated using operator new. new is followed by a data type specifier and, if a sequence of more than one element is required, the number of these within brackets []. It returns a pointer to the beginning of the new block of memory allocated. Its syntax is:
 pointer = new type
 pointer = new type [number_of_elements]
- The first expression is used to allocate memory to contain one single element of a type. The second one is used to allocate a block (an array) of elements of a type, where number_of_elements is an integer value representing the amount of these.

Example

```
int * foo;
```

```
foo = new int [5];
```

- The system dynamically allocates space for five elements of type int and returns a pointer to the first element of the sequence, which is assigned to foo (a pointer). Therefore, foo now points to a valid block of memory with space for five elements of type int.



- Here, foo is a pointer, and thus, the first element pointed to by foo can be accessed either with the expression foo[0] or the expression *foo (both are equivalent). The second element can be accessed either with foo[1] or *(foo+1), and so on...

- There is a substantial difference between declaring a normal array and allocating dynamic memory for a block of memory using new. The most important difference is that the size of a regular array needs to be a constant expression, and thus its size has to be determined at the moment of designing the program, before it is run, whereas the dynamic memory allocation performed by new allows to **assign memory during runtime** using any variable value as size.
- The dynamic memory requested by our program is allocated by the system from the memory heap. However, computer memory is a limited resource, and it can be exhausted. Therefore, there are no guarantees that all requests to allocate memory using operator new are going to be granted by the system.

- C++ provides two standard mechanisms to check if the allocation was successful:
- One is by handling exceptions. Using this method, an exception of type `bad_alloc` is thrown when the allocation fails.
- This exception method is the method used by default by `new`, and is the one used in a declaration like:
`foo = new int [5]; // if allocation fails, an exception is thrown`
- The other method is known as `nothrow`, and what happens when it is used is that when a memory allocation fails, instead of throwing a `bad_alloc` exception or terminating the program, the pointer returned by `new` is a null pointer, and the program continues its execution normally.

- This method can be specified by using a special object called nothrow, declared in header <new>, as argument for new:

```
foo = new (nothrow) int [5];
```

- In this case, if the allocation of this block of memory fails, the failure can be detected by checking if foo is a null pointer:

```
int * foo;
```

```
foo = new (nothrow) int [5];
```

```
if (foo == nullptr) {
```

```
    // error assigning memory. Take measures.
```

```
}
```

- memory allocated dynamically is only needed during specific periods of time within a program; once it is no longer needed, it can be freed so that the memory becomes available again for other requests of dynamic memory.
- This is the purpose of operator delete, whose syntax is:
 delete pointer;
 delete[] pointer;
- The first statement releases the memory of a single element allocated using new, and the second one releases the memory allocated for arrays of elements using new and a size in brackets ([])